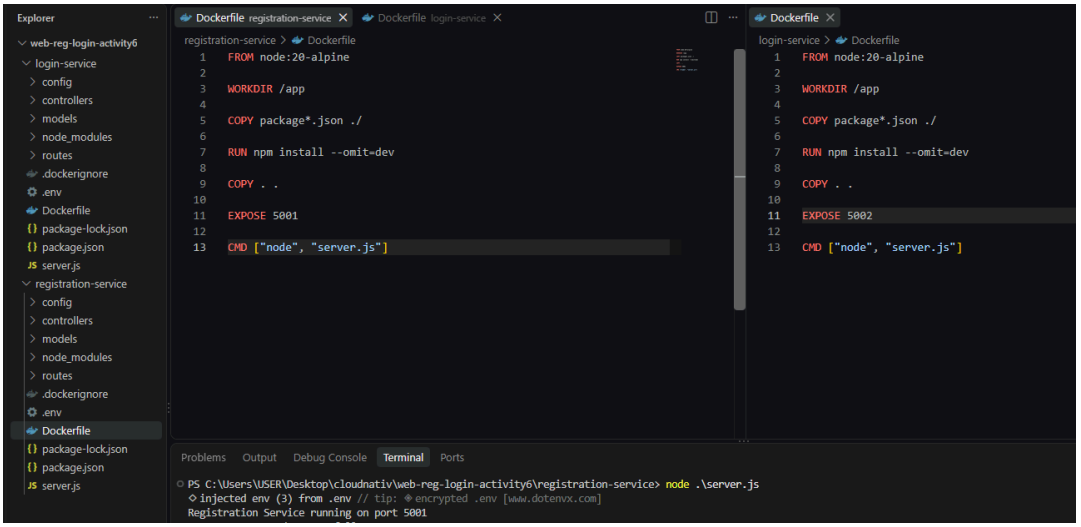
Create 2 Microservice (Reg, Login) Containers using Docker Compose

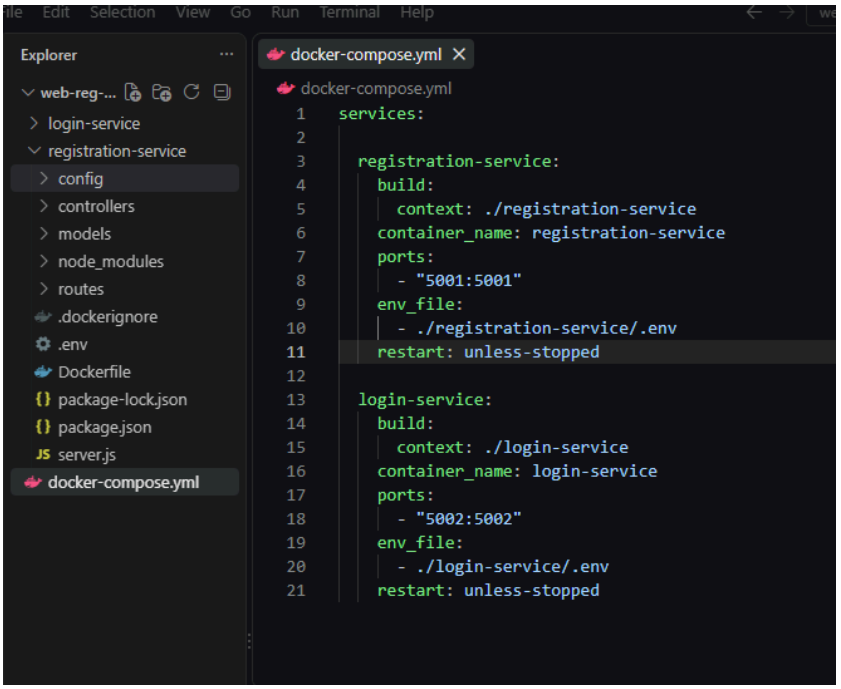
By: Kiricheat Meas

=====

2. Add Dockerfile inside each of your microservice folder—2 Dockerfiles Code Screenshot



4. Create Docker Compose File inside project folder outside each microservice folder with Port Mapping —Dockercompose file Code Screenshot

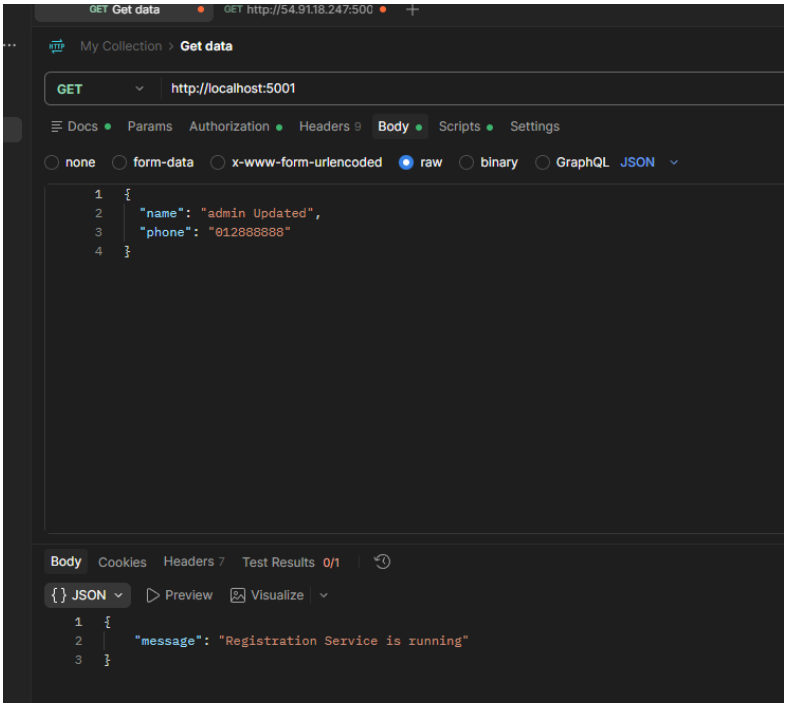


5. Run Docker Compose. - Command Screenshot

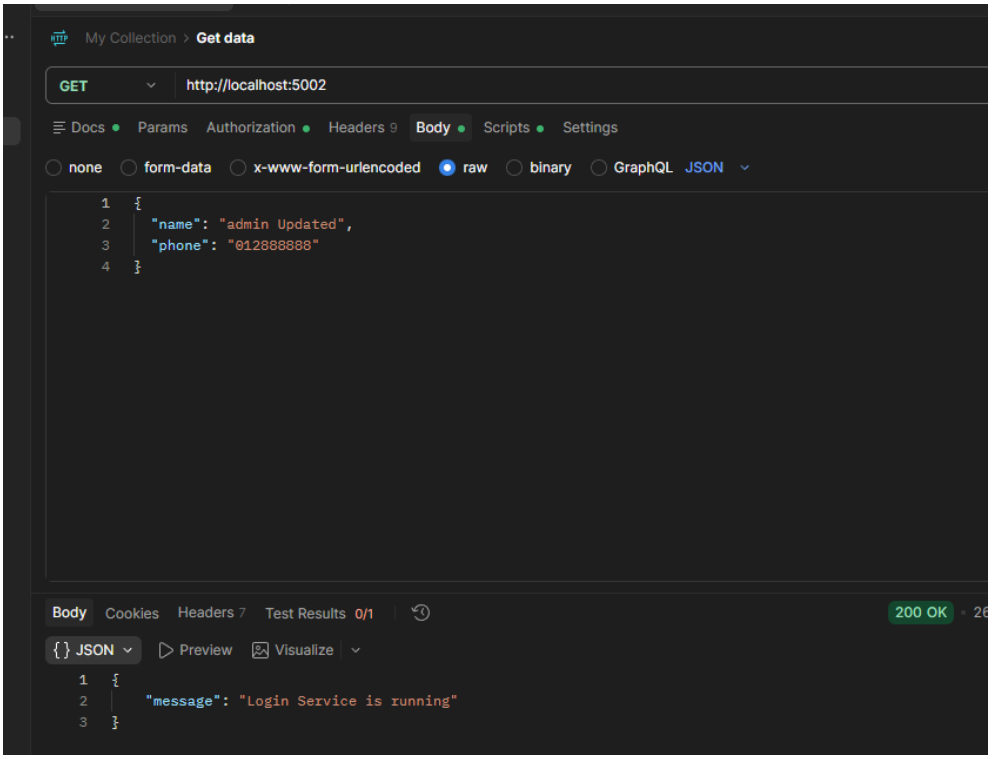
```
What's next:
    Debug this Compose error with Gordon —>docker ai "help me fix this compose error"
PS C:\Users\USER\Desktop\cloudnativ\web-reg-login-activity6> docker compose up --build -d
[+] Building 3.6s (4/5)
 => [internal] load local bake definitions
 => => reading from stdin 1.28kB
 => [login-service internal] load build definition from Dockerfile
 => => transferring dockerfile: 185B
 => [registration-service internal] load build definition from Dockerfile
 => => transferring dockerfile: 185B
 => [login-service internal] load metadata for docker.io/library/node:20-alpine
 => [auth] library/node:pull token for registry-1.docker.io
```

7. Access your APIs using POSTMAN —Postman Screenshot with URL

Registration



Login

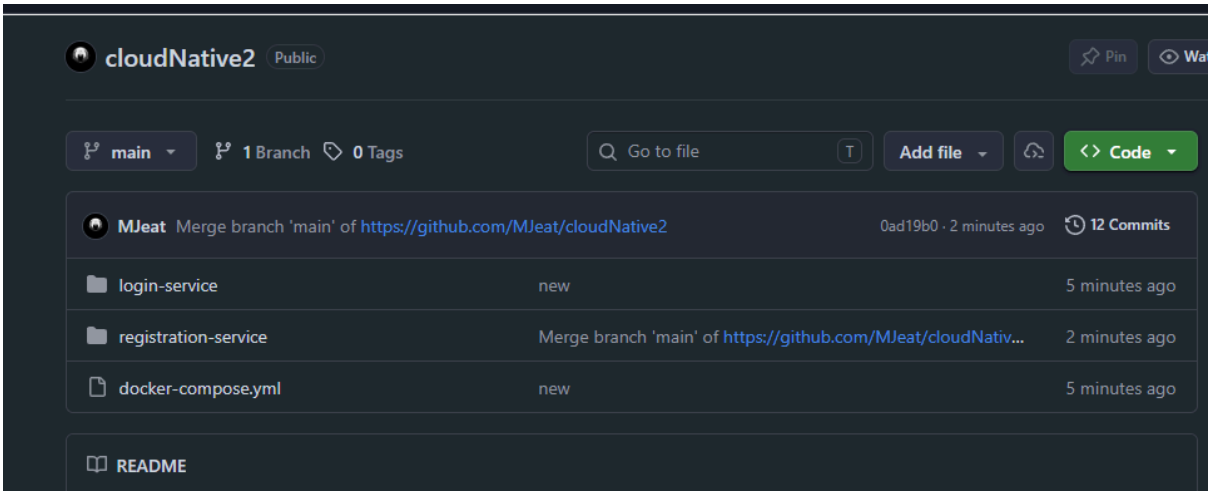



```
PS C:\Users\USER\Desktop\cloudnativ\web-reg-login-activity6> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS              PORTS                                              NAMES
50a8f2a1a5c2   25a301d8b3f7   "docker-entrypoint.s…"   About a minute ago   Up About a minute   0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp        login-service
d4716c5f387b   13460e20e8e5   "docker-entrypoint.s…"   About a minute ago   Up About a minute   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp        registration-service
PS C:\Users\USER\Desktop\cloudnativ\web-reg-login-activity6>
```

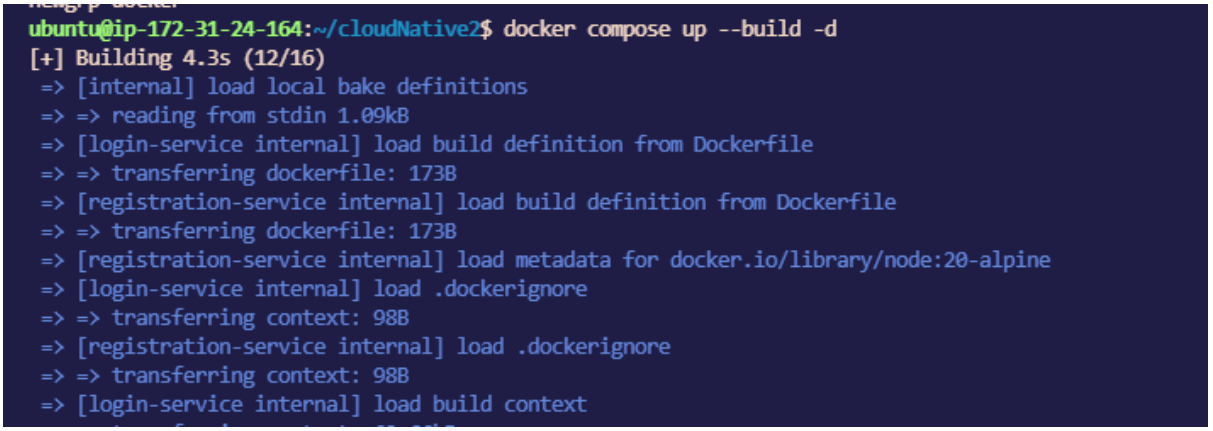
8. DOWN ALL CONTAINERS USING DOCKER COMPOSE COMMAND—Command Screenshot

```
 ✓ Container login-service                        Started
PS C:\Users\USER\Desktop\cloudnativ\web-reg-login-activity6> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS              PORTS                                              NAMES
50a8f2a1a5c2   25a301d8b3f7   "docker-entrypoint.s…"   About a minute ago   Up About a minute   0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp        login-service
d4716c5f387b   13460e20e8e5   "docker-entrypoint.s…"   About a minute ago   Up About a minute   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp        registration-service
PS C:\Users\USER\Desktop\cloudnativ\web-reg-login-activity6> docker compose down
[+] down 3/3
 ✓ Container login-service                Removed
 ✓ Container registration-service         Removed
 ✓ Network web-reg-login-activity6_default Removed
PS C:\Users\USER\Desktop\cloudnativ\web-reg-login-activity6> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\Users\USER\Desktop\cloudnativ\web-reg-login-activity6>
```

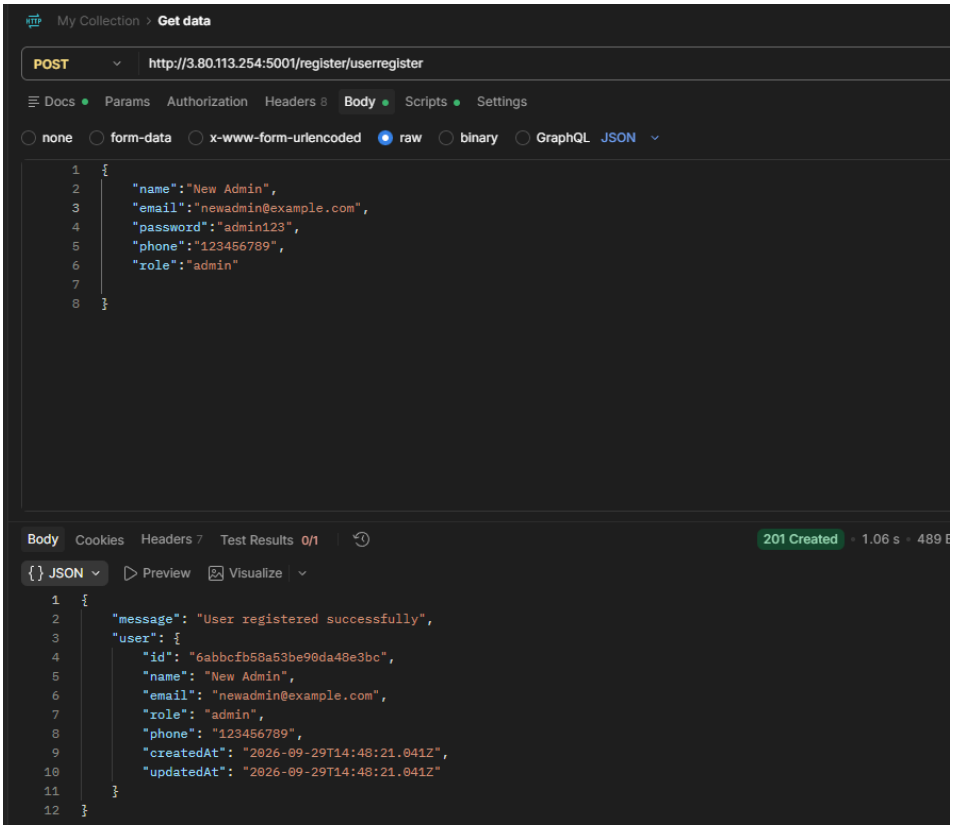
9. Push all your code in the GitHub - GitHub Screenshot

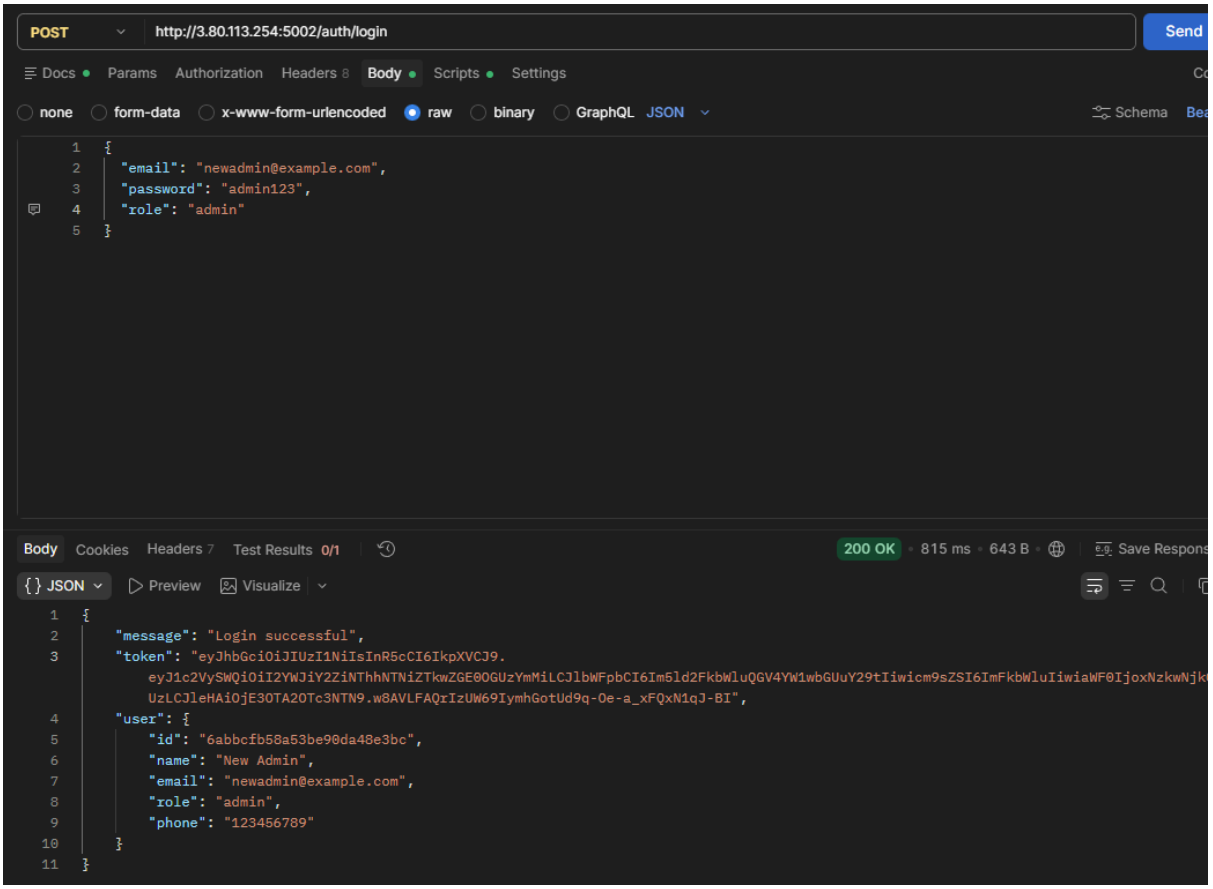


11. Run dockercompose inside your EC2. - EC2 Command line Screenshot



12. Call your registration API, Login API using Postman. - Postman Screenshot with URL + MongoDB Screenshot


My Collection > Get data
POST
http://3.80.113.254:5001/register/userregister
Docs Params Authorization Headers 8 Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL JSON
{
"name":"New Admin",
"email":"newadmin@example.com",
"password":"admin123",
"phone":"123456789",
"role":"admin"
}
Body Cookies Headers 7 Test Results 0/1
201 Created 1.06 s
{ } JSON Preview Visualize
{
"message": "User registered successfully",
"user": {
"id": "6abbcfb58a53be90da48e3bc",
"name": "New Admin",
"email": "newadmin@example.com",
"role": "admin",
"phone": "123456789",
"createdAt": "2026-09-29T14:48:21.041Z",
"updatedAt": "2026-09-29T14:48:21.041Z"
}
}


POST
http://3.80.113.254:5002/auth/login
Send
Docs Params Authorization Headers 8 Body Scripts Settings
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema
{
"email": "newadmin@example.com",
"password": "admin123",
"role": "admin"
}
Body Cookies Headers 7 Test Results 0/1
200 OK 815 ms 643 B
Save Respons
{ } JSON Preview Visualize
{
"message": "Login successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJ1c2VySWQiOiI2YWJiY2ZiNThhNTNiZTkwZGE0OGUzYmMiLCJlbWFpbCI6Im5ld2FkbWluQGV4YW1wbGUuY29tIiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzkwNjk
UzLCJleHAiOjE3OTA2OTc3NTN9.w8AVLFAQrIzUW69IymhGotUd9q-Oe-a_xFQxN1qJ-BI",
"user": {
"id": "6abbcfb58a53be90da48e3bc",
"name": "New Admin",
"email": "newadmin@example.com",
"role": "admin",
"phone": "123456789"
}
}

```
_id: ObjectId('6abbcfb58a53be90da48e3bc')
name : "New Admin"
email : "newadmin@example.com"
password : "$2b$10$hPO09fLTH9NBzE2ifHB30.CTu6UQ.kZsyPg9BuPw8Rfyn/XWHw52."
role : "admin"
phone : "123456789"
createdAt : ISODate('2026-09-29T14:48:21.041+00:00') September 29, 2026 at 2:48:21 PM UTC
updatedAt : ISODate('2026-09-29T14:48:21.041+00:00') September 29, 2026 at 2:48:21 PM UTC
__v : 0
```


===X===